

Experiment: Install Git, Create a GitHub Account and Execute Various Git Operations

Aim

To install Git, create a GitHub account, configure Git, connect a local repository with GitHub, and perform various Git operations such as initialization, staging, committing, branching, merging, pushing, pulling, cloning, and viewing repository history.

Introduction

Git is a distributed version control system developed by Linus Torvalds to efficiently track changes in source code. Git allows multiple developers to collaborate on the same project while maintaining a complete history of modifications. GitHub is a cloud-based hosting platform that stores Git repositories and provides collaboration features including pull requests, issue tracking, code review, and project management. Git and GitHub together form the foundation of modern software development and DevOps workflows. Git is a distributed version control system developed by Linus Torvalds to efficiently track changes in source code. Git allows multiple developers to collaborate on the same project while maintaining a complete history of modifications. GitHub is a cloud-based hosting platform that stores Git repositories and provides collaboration features including pull requests, issue tracking, code review, and project management. Git and GitHub together form the foundation of modern software development and DevOps workflows. Git is a distributed version control system developed by Linus Torvalds to efficiently track changes in source code. Git allows multiple developers to collaborate on the same project while maintaining a complete history of modifications. GitHub is a cloud-based hosting platform that stores Git repositories and provides collaboration features including pull requests, issue tracking, code review, and project management. Git and GitHub together form the foundation of modern software development and DevOps workflows. Git is a distributed version control system developed by Linus Torvalds to efficiently track changes in source code. Git allows multiple developers to collaborate on the same project while maintaining a complete history of modifications. GitHub is a cloud-based hosting platform that stores Git repositories and provides collaboration features including pull requests, issue tracking, code review, and project management. Git and GitHub together form the foundation of modern software development and DevOps workflows. Git is a distributed version control system developed by Linus Torvalds to efficiently track changes in source code. Git allows multiple developers to collaborate on the same project while maintaining a complete history of modifications. GitHub is a cloud-based hosting platform that stores Git repositories and provides collaboration features including pull requests, issue tracking, code review, and project management. Git and GitHub together form the foundation of modern software development and DevOps workflows.

System Requirements

Operating System: Windows/Linux/macOS.

Software: Git, GitHub Account, Internet Connection, VS Code or any text editor.

Procedure

1. Download Git from the official website and install it.
2. Create a GitHub account.
3. Configure Git using username and email.
4. Create a local repository using **git init**.
5. Create project files.
6. Add files using **git add .**
7. Commit changes using **git commit -m**.
8. Create a GitHub repository.
9. Connect local repository using remote URL.
10. Push project to GitHub.
11. Perform pull, clone, branch and merge operations.

Git Commands

git --version – Check Git installation.
git config --global user.name "Your Name"
git config --global user.email "email@example.com"
git init
git status
git add .
git commit -m "Initial Commit"
git branch feature
git checkout feature
git merge feature
git remote add origin URL
git push -u origin main
git pull origin main
git clone URL
git log

Common Git Workflow

Create Repository → Initialize Git → Add Files → Commit Changes → Create GitHub Repository → Connect Remote Repository → Push Code → Collaborate → Pull Latest Changes → Merge Branches → Deploy Application. Create Repository → Initialize Git → Add Files → Commit Changes → Create GitHub Repository → Connect Remote Repository → Push Code → Collaborate → Pull Latest Changes → Merge Branches → Deploy Application. Create Repository → Initialize Git → Add Files → Commit Changes → Create GitHub Repository → Connect Remote Repository → Push Code → Collaborate → Pull Latest Changes → Merge Branches → Deploy Application. Create Repository → Initialize Git → Add Files → Commit Changes → Create GitHub Repository → Connect Remote Repository → Push Code → Collaborate → Pull Latest Changes → Merge Branches → Deploy Application. Create Repository → Initialize Git → Add Files → Commit Changes → Create GitHub Repository → Connect Remote Repository → Push Code → Collaborate → Pull Latest Changes → Merge Branches → Deploy Application. Create Repository → Initialize Git → Add Files → Commit Changes → Create GitHub Repository → Connect Remote Repository → Push Code → Collaborate → Pull Latest Changes → Merge Branches → Deploy Application.

Explanation of Operations

Git initialization creates a repository. Staging prepares files for commit. Commit permanently records changes. Branching enables independent development. Merging combines different branches. Push uploads commits to GitHub. Pull downloads recent updates from GitHub. Clone creates a local copy of a remote repository. Git log displays commit history. Git initialization creates a repository. Staging prepares files for commit. Commit permanently records changes. Branching enables independent development. Merging combines different branches. Push uploads commits to GitHub. Pull downloads recent updates from GitHub. Clone creates a local copy of a remote repository. Git log displays commit history. Git initialization creates a repository. Staging prepares files for commit. Commit permanently records changes. Branching enables independent development. Merging combines different branches. Push uploads commits to GitHub. Pull downloads recent updates from GitHub. Clone creates a local copy of a remote repository. Git log displays commit history. Git initialization creates a repository. Staging prepares files for commit. Commit permanently records changes. Branching enables independent development. Merging combines different branches. Push uploads commits to GitHub. Pull downloads recent updates from GitHub. Clone creates a local copy of a remote repository. Git log displays commit history. Git initialization creates a repository. Staging prepares files for commit. Commit permanently records changes. Branching enables independent development. Merging combines different branches. Push uploads commits to GitHub. Pull downloads recent updates from GitHub. Clone creates a local copy of a remote repository. Git log displays commit history. Git initialization creates a repository. Staging prepares files for commit. Commit permanently records changes. Branching enables independent development. Merging combines different branches. Push uploads commits to GitHub. Pull downloads recent updates from GitHub. Clone creates a local copy of a remote repository. Git log displays commit history. Git initialization creates a repository. Staging prepares files for commit. Commit permanently records changes. Branching enables independent development. Merging combines different branches. Push uploads commits to GitHub. Pull downloads recent updates from GitHub. Clone creates a local copy of a remote repository. Git log displays commit history.

Advantages

Git provides distributed version control, faster collaboration, rollback capability, branching support, complete project history, conflict management, offline development, easy backup, and seamless integration with GitHub and CI/CD pipelines. Git provides distributed version control, faster collaboration, rollback capability, branching support, complete project history, conflict management, offline development, easy backup, and seamless integration with GitHub and CI/CD pipelines. Git provides distributed version control, faster collaboration, rollback capability, branching support, complete project history, conflict management, offline development, easy backup, and seamless integration with GitHub and CI/CD pipelines. Git provides distributed version control, faster collaboration, rollback capability, branching support, complete project history, conflict management, offline development, easy backup, and seamless integration with GitHub and CI/CD pipelines. Git provides distributed version control, faster collaboration, rollback capability, branching support, complete project history, conflict management, offline development, easy backup, and seamless integration with GitHub and CI/CD pipelines. Git provides distributed version control, faster collaboration, rollback capability, branching support, complete project history, conflict management, offline development, easy backup, and seamless integration with GitHub and CI/CD pipelines. Git provides distributed version control, faster collaboration, rollback capability, branching support, complete project history, conflict management, offline development, easy backup, and seamless integration with GitHub and CI/CD pipelines.

Applications

Git is widely used in software development, web development, machine learning projects, DevOps automation, open-source projects, mobile application development, cloud-native applications, research collaboration, and enterprise software engineering. Git is widely used in software development, web development, machine learning projects, DevOps automation, open-source projects, mobile application development, cloud-native applications, research collaboration, and enterprise software engineering. Git is widely used in software development, web development, machine learning projects, DevOps automation, open-source projects, mobile application development, cloud-native applications, research collaboration, and enterprise software engineering. Git is widely used in software development, web development, machine learning projects, DevOps automation, open-source projects, mobile application development, cloud-native applications, research collaboration, and enterprise software engineering. Git is widely used in software development, web development, machine learning projects, DevOps automation, open-source projects, mobile application development, cloud-native applications, research collaboration, and enterprise software engineering. Git is widely used in software development, web development, machine learning projects, DevOps automation, open-source projects, mobile application development, cloud-native applications, research collaboration, and enterprise software engineering.

Result

Git was successfully installed, a GitHub account was created, and various Git operations such as initialization, staging, committing, branching, merging, pushing, pulling, cloning, and viewing logs were successfully performed.

Conclusion

Git and GitHub provide an efficient platform for version control and collaborative software development. Understanding Git commands and workflows is essential for modern software engineering, DevOps practices, and professional software development. Git and GitHub provide an efficient platform for version control and collaborative software development. Understanding Git commands and workflows is essential for modern software engineering, DevOps practices, and professional software development. Git and GitHub provide an efficient platform for version control and collaborative software development. Understanding Git commands and workflows is essential for modern software engineering, DevOps practices, and professional software development. Git and GitHub provide an efficient platform for version control and collaborative software development. Understanding Git commands and workflows is essential for modern software engineering, DevOps practices, and professional software development.